

Git, GitHub & Bitbucket - Assignment and Notes

1. What is Git?

Git is a distributed version control system. It tracks changes in files and lets developers save versions of their work, create branches, compare changes and collaborate safely.

Explanation: Git works on your computer. You can use Git even without GitHub or Bitbucket.

2. What is GitHub?

GitHub is an online platform for hosting Git repositories. It adds collaboration features such as pull requests, issues, Actions, project boards and repository access controls.

Explanation: GitHub stores a remote copy of your Git repository and makes collaboration easier.

3. What is Bitbucket?

Bitbucket is another platform for hosting Git repositories. It is commonly used by teams that work with Atlassian tools such as Jira and Confluence.

Explanation: The basic Git commands are the same whether the remote repository is on GitHub or Bitbucket.

5. Git vs GitHub vs Bitbucket

Tool	Type	Main Purpose	Works Locally?
Git	Version control system	Track file changes and versions	Yes
GitHub	Git hosting platform	Host repositories and collaborate online	No - online service
Bitbucket	Git hosting platform	Host repositories and collaborate online	No - online service

6. Install and Check Git

```
git --version
```

Explanation: Shows the installed Git version. Example output: git version 2.51.0.windows.1

7. Configure Git

```
git config --global user.name "Abdul Hafeez"
git config --global user.email "your-email@example.com"
```

Explanation: Git stores your name and email inside commits. Use the email associated with your Git hosting account when appropriate.

```
git config --global --list
```

Explanation: Displays your global Git configuration.

8. Create a Practice Project

```
mkdir git-practice
cd git-practice
```

Explanation: Creates a new folder and moves into it.

```
echo "# Git Practice" > README.md
```

Explanation: Creates a simple README file. On Windows PowerShell, this command also works.

9. Initialize a Git Repository

```
git init
```

Explanation: Creates a hidden .git folder. The current project is now a Git repository.

10. Check Repository Status

```
git status
```

Explanation: Shows modified, untracked and staged files. You should use `git status` frequently.

11. Add Files to Staging

```
git add README.md
```

Explanation: Adds only `README.md` to the staging area.

```
git add .
```

Explanation: Stages all changed and new files in the current project. Review `git status` before committing.

12. Create a Commit

```
git commit -m "Add project README"
```

Explanation: A commit saves a version of the staged files with a message describing the change.

13. View Commit History

```
git log
```

Explanation: Shows commit history with commit IDs, author, date and message.

```
git log --oneline
```

Explanation: Shows a shorter one-line version of the history. Example: `a1b2c3d Add project README`

14. Modify and Compare Files

```
echo "Learning Git step by step." >> README.md  
git diff
```

Explanation: `git diff` shows changes in your working directory that have not yet been staged.

```
git add README.md  
git diff --staged
```

Explanation: Shows changes that are staged and ready for the next commit.

15. Undo Common Changes

```
git restore README.md
```

Explanation: Discards unstaged changes in `README.md`. Use carefully because those local changes are lost.

```
git restore --staged README.md
```

Explanation: Removes `README.md` from the staging area but keeps the file changes.

16. Create and Use Branches

```
git branch feature-about
```

Explanation: Creates a new branch named `feature-about`.

```
git switch feature-about
```

Explanation: Moves to the `feature-about` branch.

```
git switch -c feature-contact
```

Explanation: Creates and switches to a new branch in one command.

```
git branch
```

Explanation: Lists local branches. The current branch is marked with an asterisk.

17. Merge a Branch

```
git switch main
git merge feature-about
```

Explanation: Switches to main and merges the feature-about branch into it.

18. Delete a Finished Branch

```
git branch -d feature-about
```

Explanation: Deletes a branch after it has been merged. Git protects you from deleting unmerged work with -d.

19. Create a .gitignore File

```
# Dependencies
node_modules/

# Environment files
.env
.env.local

# Logs
*.log

# Build folders
dist/
build/
.next/
```

Explanation: .gitignore tells Git which files and folders should not be tracked. Never commit passwords or secret keys.

20. Connect a Local Project to GitHub

First create an empty repository on GitHub. Then run commands similar to these:

```
git remote add origin https://github.com/USERNAME/git-practice.git
git branch -M main
git push -u origin main
```

Explanation: origin is the usual name for the remote repository. -u connects your local main branch to origin/main so future pushes can use git push.

21. Clone a GitHub Repository

```
git clone https://github.com/USERNAME/git-practice.git
```

Explanation: Downloads the repository, its files and Git history to your computer.

22. Pull Remote Changes

```
git pull origin main
```

Explanation: Downloads remote changes and integrates them into your current branch. Pull before pushing when other collaborators may have changed the repository.

23. Push Local Changes

```
git add .
git commit -m "Update project notes"
git push origin main
```

Explanation: This is a common workflow: stage changes, commit them, then upload the commit to the remote repository.

24. Check Remote Repositories

```
git remote -v
```

Explanation: Shows remote names and their fetch/push URLs.

25. GitHub Feature Branch Workflow

```
git switch -c feature-login
# edit files
git add .
git commit -m "Add login page"
git push -u origin feature-login
```

Explanation: Push the feature branch to GitHub. Then open a Pull Request on GitHub to review and merge the branch into main.

26. What is a Pull Request?

A Pull Request (PR) is a request to review and merge changes from one branch into another branch. Teams use pull requests for code review, discussion, automated checks and approval before merging.

27. GitHub Issues

Issues are used to track bugs, tasks, ideas and feature requests. A good issue normally includes a clear title, description, steps to reproduce when relevant and expected result.

28. Bitbucket Remote Workflow

Create an empty repository in Bitbucket and copy its Git URL. Then use normal Git commands:

```
git remote add origin https://USERNAME@bitbucket.org/WORKSPACE/git-practice.git
git branch -M main
git push -u origin main
```

Explanation: The Git workflow is almost the same as GitHub. Only the remote URL and platform interface are different.

29. Clone from Bitbucket

```
git clone https://USERNAME@bitbucket.org/WORKSPACE/git-practice.git
```

Explanation: Downloads the Bitbucket repository to your computer.

30. Bitbucket Branch and Pull Request Workflow

```
git switch -c feature-menu
# make changes
git add .
git commit -m "Add menu page"
git push -u origin feature-menu
```

Explanation: After pushing, create a Pull Request in Bitbucket to merge feature-menu into main.

31. GitHub and Bitbucket Comparison

Feature	GitHub	Bitbucket
Git repository hosting	Yes	Yes
Pull requests	Yes	Yes
Issues	Strong built-in support	Available through platform/team workflow
CI/CD	GitHub Actions	Bitbucket Pipelines
Atlassian integration	Available	Very strong with Jira/Confluence
Open-source visibility	Very popular	More commonly seen in private/team environments

32. Merge Conflict Example

A merge conflict happens when Git cannot automatically decide how to combine changes from two branches.

```
git merge feature-home
```

Explanation: If both branches changed the same lines differently, Git may stop and mark the conflicted file.

```
<<<<<< HEAD
Welcome to the main page
=====
Welcome to our home page
>>>>>> feature-home
```

Explanation: Edit the file manually, keep the correct final text, remove the conflict markers, then stage and commit the resolved file.

```
git add README.md
git commit -m "Resolve README merge conflict"
```

33. Git Stash

```
git stash
```

Explanation: Temporarily stores uncommitted changes so you can switch branches with a clean working directory.

```
git stash list
git stash pop
```

Explanation: Lists saved stashes and restores the most recent stash.

34. Tags

```
git tag v1.0.0
git push origin v1.0.0
```

Explanation: Tags are useful for marking important versions or releases.

35. Revert a Commit Safely

```
git log --oneline
git revert COMMIT_ID
```

Explanation: `git revert` creates a new commit that reverses an earlier commit. It is generally safer than rewriting shared history.

36. Helpful Daily Git Workflow

```
git pull
git status
git switch -c feature-name
# make changes
git add .
git commit -m "Describe the change"
git push -u origin feature-name
```

Explanation: Then create a Pull Request, get it reviewed, merge it, return to main and pull the latest changes.

37. Useful Git Commands - Quick Reference

Command	Purpose
<code>git init</code>	Initialize a repository
<code>git status</code>	Check file status
<code>git add .</code>	Stage changes
<code>git commit -m "message"</code>	Create a commit
<code>git log --oneline</code>	View short history
<code>git branch</code>	List branches
<code>git switch branch</code>	Change branch
<code>git switch -c branch</code>	Create and switch branch
<code>git merge branch</code>	Merge a branch
<code>git remote -v</code>	Show remotes
<code>git clone URL</code>	Clone a remote repository
<code>git pull</code>	Download/integrate remote changes
<code>git push</code>	Upload commits
<code>git diff</code>	Show unstaged differences
<code>git stash</code>	Temporarily save local changes
<code>git tag</code>	Create/list tags

End of Git, GitHub & Bitbucket Assignment Notes